Aidan Rohm

Artificial Intelligence - CISC 6525

Professor Lyons

19 February, 2026

Homework Assignment 2

1. Iterative Deepening Question
    a. Order of nodes visited → Assuming that node 13 is the goal (since it is the "deepest" node)
        i. Depth: 0, Node Order: 1
        ii. Depth: 1, Node Order: 1, 2, 3, 4
        iii. Depth: 2, Node Order: 1, 2, 5, 6, 3, 7, 4, 8, 9
        iv. Depth: 3, Node Order: 1, 2, 5, 6, 10, 11, 3, 7, 12, 13
    b. Explanation:
        i. Iterative Deepening Search is a unique case of Depth First Search that simply constrains the depth that it can go before iterating to the next possible depth. The search algorithm combines the space efficiency of Depth First Search with the unit cost optimality of Breadth First Search. The specific observed pattern occurs because Iterative Deepening Search first explores all of the shallow nodes. As the frontier is pushed deeper, the algorithm searches as deep as it can first before back tracking and expanding the next shallowest node. Every new depth limit causes the upper "levels" to be re-expanded before continuing to go deeper. The

re-expansion feature is what promotes a repeated appearance of nodes even if they were previously explored.

2. A* Search Question: Timisoara → Bucharest
    a. (T, 0+329=329)
    b. Expand T → Only node, by default it has the lowest estimated cost
    c. (A, 118+366=484) (L, 111+244=355)
    d. Expand L → Has the lowest estimated cost of all unexplored remaining nodes
    e. (A, 118+366=484) (M, 181+241=422)
    f. Expand M → Has the lowest estimated cost of all unexplored remaining nodes
    g. (A, 118+366=484) (D, 256+242=498)
    h. Expand A → Has the lowest estimated cost of all unexplored remaining nodes
    i. (D, 256+242=498) (S, 258+253=511) (Z, 193+374=567)
    j. Expand D → Has the lowest estimated cost of all unexplored remaining nodes
    k. (C, 376+160=536) (S, 258+253=511) (Z, 193+374=567)
    l. Expand S → Has the lowest estimated cost of all unexplored remaining nodes
    m. (R, 338+193=531) (F, 357+176=533) (C, 376+160=536) (Z, 193+374=567)
    n. Expand R → Has the lowest estimated cost of all unexplored remaining nodes
    o. (F, 357+176=533) (P, 435+100=535) (C, 376+160=536) (Z, 193+374=567) (C, 486+160=646)
    p. Expand F → Has the lowest estimated cost of all unexplored remaining nodes
    q. (B, 568+0=568) (P, 435+100=535) (C, 376+160=536) (Z, 193+374=567) (C, 486+160=646)
    r. Expand P → Has the lowest estimated cost of all unexplored remaining nodes

s. (B, 568+0=568) (B, 536+0) (C, 376+160=536) (Z, 193+374=567) (C, 486+160=646)

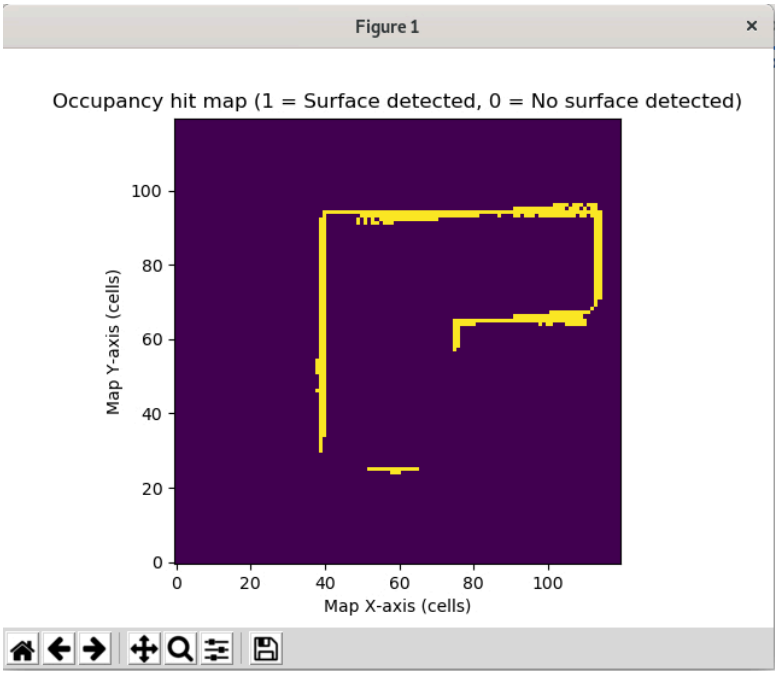
t. Expand B → Tied lowest estimated cost with C so it selects either. B=Bucharest and is the goal, so it is finished

u. Path is:

i. Timisoara → Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest for a total cost of 536

3. GoTo.py question

a. The code has been attached as "rohm_h2q3.py" and contains all of my modifications for the goto.py function. The below attached screenshot shows the map that the robot produced after moving to position (1,1) from around position (-2, -2). This starting position close to (-2, -2) was simply a result of repeated experimentation.

b. 

c. I made three major changes to the original goto.py program. First it was important to create all necessary subscriptions for the laser rangefinder data. The primary function here was the laser callback function. In this, a loop iterates through the laser ranges keeping only valid readings. It then converts each reading into a world map point before converting it into a grid index. The numpy array is then marked with a 1 for this index to show that this position is occupied. Another function takes a single laser reading and the robot's pose and uses them to create (X, Y) coordinates for the detected surface. Finally, I created a world-to-map function that converts a point in meters into numpy array indices (row, col) based on the map resolution that I declare globally. The map is anchored based on the robot's starting position.